

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

Error Analysis Task

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15% Assessment

Tool Weightage: 30% Duration:

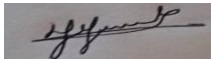
30 minutes

Marks: 25

Code of Conduct:

I D.Chinna Moulali/192425278 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

QUESTIONS:

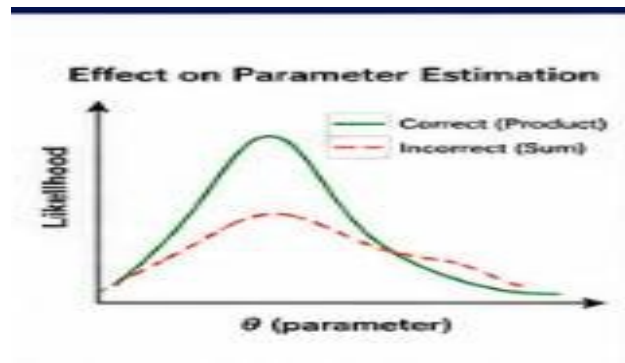
Total Marks:25

1. Error Analysis in Maximum Likelihood Estimation (MLE)

A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.

Tasks:

- Identify the conceptual error in the likelihood formulation.
 - Explain how this mistake affects parameter estimation.
 - Suggest the correct mathematical formulation and reasoning.
 - How would using log-likelihood fix computational issues?
- a) Identify the conceptual error in the likelihood formulation.**
- The likelihood was incorrectly written as the **sum of probabilities** instead of the **product of probabilities** of all independent observations.
 - For independent samples, likelihood is obtained by multiplying the probability of each observation.



b) Explain how this mistake affects parameter estimation.

- Summing probabilities does not represent the joint probability of the dataset.
- The optimization objective becomes mathematically incorrect.
- Estimated parameters will not maximize the true likelihood, leading to biased and inaccurate predictions even with large datasets.

c) Suggest the correct mathematical formulation and reasoning.

For independent observations:

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

where:

- $L(\theta)$ = likelihood
- θ = model parameters
- x_i = observations

The product correctly represents the probability of observing the complete dataset assuming independence.

d) How would using log-likelihood fix computational issues?

The log-likelihood is

$$\log L(\theta) = \sum_{i=1}^n \log P(x_i | \theta)$$

Advantages:

- Converts multiplication into addition.
- Prevents numerical underflow caused by multiplying many small probabilities.
- Simplifies differentiation and optimization.
- Produces the same optimal parameter estimates because the logarithm is a monotonic function.

2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.

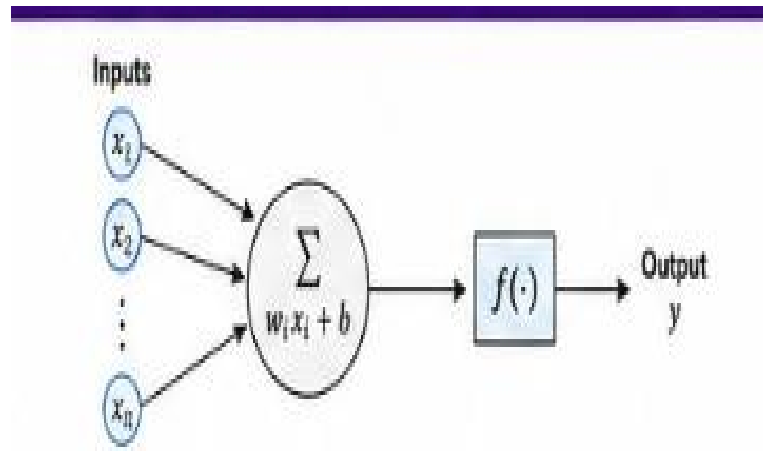
Tasks:

- List possible implementation errors in the learning algorithm.
- Analyze how incorrect learning rate or weight updates affect convergence.
- Identify dataset-related issues that may cause this problem.
- Propose modifications to ensure convergence.

a) List possible implementation errors.

- Incorrect weight update formula.

- Missing or incorrect bias term.
- Wrong prediction threshold.
- Labels encoded incorrectly (0/1 instead of $-1/+1$ when required).
- Incorrect stopping criterion.
- Programming errors in updating weights.



b) Analyze how incorrect learning rate or weight updates affect convergence.

- Very large learning rate causes oscillations and instability.
- Very small learning rate slows convergence.
- Incorrect update direction moves weights away from the optimal solution.
- Missing updates for misclassified samples prevents learning.

c) Identify dataset-related issues.

- Features are not normalized.
- Data may contain mislabeled examples.
- Hidden noise or outliers.
- Dataset may not actually be linearly separable despite assumptions.

d) Propose modifications.

- Normalize feature values.
- Verify correct implementation of

$$w = w + \eta(y - \hat{y})x$$

- Choose an appropriate learning rate.
- Shuffle training data each epoch.
- Remove noisy or mislabeled samples.
- If data is not linearly separable, use kernel methods or logistic regression.

3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.

Tasks:

- Identify possible mathematical or coding errors in gradient computation.
- Explain the role of activation functions in gradient flow.
- Analyze how vanishing or exploding gradients affect training.
- Suggest techniques to fix the issue (e.g., normalization, initialization).

a) Identify possible mathematical or coding errors.

- Incorrect derivative of activation function.
- Wrong chain rule implementation.

- Incorrect sign of gradient update.
- Gradient indexing mistakes.
- Parameters updated before completing backpropagation.
- Incorrect learning rate.

b) Explain the role of activation functions.

Activation functions introduce non-linearity. Their derivatives determine gradient flow:

- **Sigmoid** may saturate and produce very small gradients.
- **Tanh** performs better but can still suffer vanishing gradients.
- **ReLU** reduces vanishing gradient issues for positive inputs.

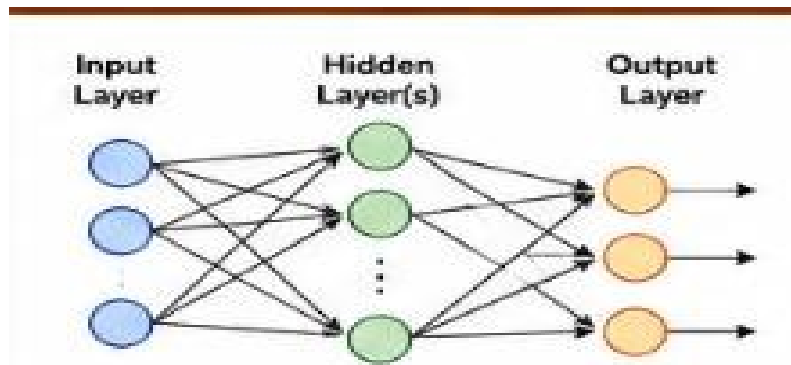
c) Analyze vanishing or exploding gradients.

Vanishing gradients

- Gradients become nearly zero.
- Early layers stop learning.
- Training becomes extremely slow.

Exploding gradients

- Gradients become excessively large.
- Weights change dramatically.
- Training becomes unstable and loss increases.



d) Suggest techniques to fix the issue.

- Xavier or He weight initialization.
- Batch Normalization.
- Gradient Clipping.
- ReLU or Leaky ReLU activation.
- Learning rate scheduling.
- Adam or RMSProp optimizer.
- Normalize input features.

4. Gradient Descent vs SGD Error Investigation

A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.

Tasks:

- Explain why batch gradient descent is slow in large datasets.
- Analyze the cause of fluctuations in SGD.
- Compare the error behavior of Mini-Batch Gradient Descent.
- Recommend the best approach and justify with error trade-offs.

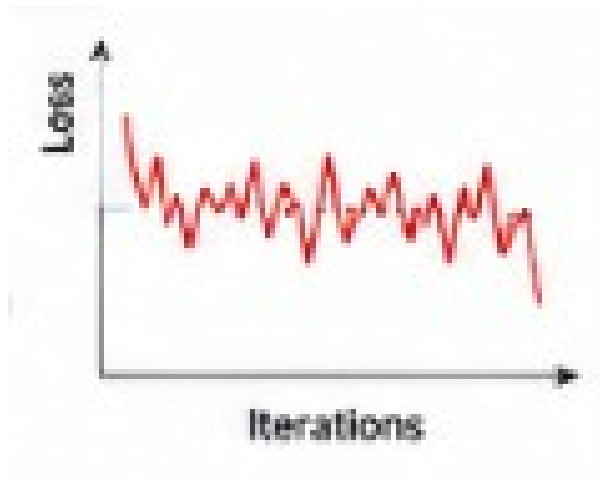
a) Explain why Batch Gradient Descent is slow.

- Computes gradients using the entire dataset before every update.
- Computational cost increases with dataset size.
- Requires significant memory.

- Each iteration takes longer.

b) Analyze the cause of fluctuations in SGD.

- Uses one sample (or very few samples) for each update.
- Gradient estimates are noisy.
- Loss fluctuates due to randomness.
- May overshoot the optimum if learning rate is high.



c) Compare Mini-Batch Gradient Descent.

Batch GD	SGD	Mini-Batch GD
Stable updates	Noisy updates	Balanced updates
Slow	Fast	Moderate speed
High computation	Low computation	Efficient computation
Smooth convergence	Fluctuating convergence	More stable convergence

Mini-batch typically uses batches of 32–256 samples.

d) Recommend the best approach.

Mini-Batch Gradient Descent is generally the preferred choice because it:

- Converges faster than Batch GD.
- Produces less noisy updates than SGD.
- Uses memory efficiently.
- Is well suited for GPU acceleration.
- Provides a good balance between convergence speed and stability.

5. Curse of Dimensionality Error Analysis

A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.

Tasks:

- Explain how high dimensionality increases model error.
- Identify signs of overfitting in this scenario.
- Analyze why distance-based learning fails in high dimensions.
- Suggest dimensionality reduction or regularization techniques to improve performance.

a) Explain how high dimensionality increases model error.

- Data becomes sparse as dimensions increase.
- More training samples are required.
- The model learns noise instead of patterns.
- Variance increases, leading to poor generalization.

b) Identify signs of overfitting.

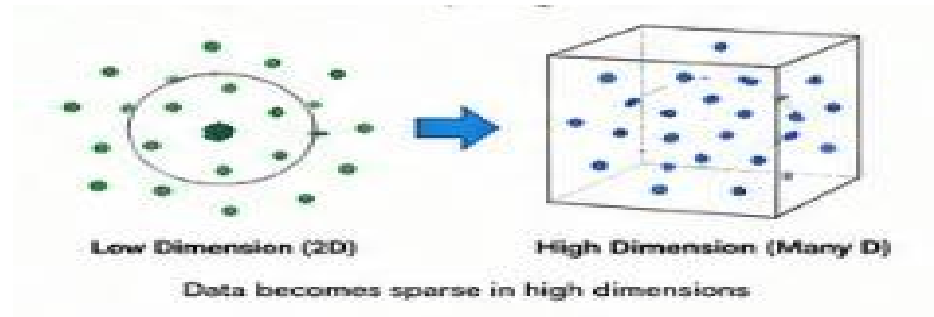
- Very high training accuracy.
- Low test accuracy.

- Large gap between training and testing performance.
- High variance across different datasets.

c) Analyze why distance-based learning fails.

- Distances between data points become increasingly similar.
- Nearest neighbors become less meaningful.
- Distance metrics lose discriminative power.
- Algorithms like KNN and K-Means perform poorly.

d) Suggest dimensionality reduction or regularization techniques.



Dimensionality Reduction

- Principal Component Analysis (PCA)
- Linear Discriminant Analysis (LDA)
- Autoencoders
- t-SNE (visualization)
- UMAP

Regularization

- L1 (Lasso)
- L2 (Ridge)
- Elastic Net
- Feature Selection
- Dropout (for neural networks)
- Early Stopping

These techniques reduce model complexity, improve generalization, and mitigate overfitting caused by high-dimensional data.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to	Applies relevant concepts	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts

		solve complex problems	correctly with minor errors		
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand